

PulseDesk

Product Requirements Document (PRD)

Support Ticket Dashboard | Full-stack Web Application

Version	1.0 - Frozen for implementation
Source	Web Application Developer Technical Assignment
Implementation constraint	4-6 hours; local execution required
Primary user	Customer support team member

Product statement

PulseDesk replaces spreadsheet-based support tracking with a simple web application that lets support staff create tickets, track status, find requests quickly, and update ticket priority or status from one responsive dashboard.

Scope principle

Build the required ticket-management workflow completely and cleanly before adding optional enhancements. The product is intentionally a focused support dashboard, not a full Zendesk-style platform.

1. Product Overview

Problem

A small company currently manages customer support requests in spreadsheets. This makes requests harder to search, prioritize, track, and update consistently.

Product goal

Give support staff one place to create, find, review, prioritize, and resolve customer support requests while keeping the data persistent and easy to understand.

Primary success outcomes

- A support agent can create a valid ticket and see it persisted in the system.
- A support agent can search, filter, sort, and paginate tickets using backend-driven querying.
- A support agent can open a ticket and update its status and priority.
- Dashboard summary counts always represent the entire dataset, independent of active filters.
- The application remains usable on desktop and mobile and handles loading, empty, and error states.

2. Users and Persona

Primary user: Support Team Member

- Receives and records customer support requests.
- Needs to quickly identify open or high-priority work.
- Searches tickets using a customer email or issue title.
- Updates ticket status and priority as work progresses.

Authentication, user roles, and a customer-facing portal are outside the required scope.

3. Scope

In scope	Out of scope
Ticket creation	Authentication and authorization
Ticket listing and detail view	Customer self-service portal
Search by title or customer email	Ticket assignment to agents
Filter by status and priority	Email/SMS notifications
Sort by creation date	File attachments
Backend pagination	Live chat / WebSockets
Update ticket status and priority	Advanced analytics and reporting
Summary counts	AI chatbot or auto-classification
Validation and API error handling	Microservices, Kafka, Redis, Kubernetes
Responsive UI and UI states	Audit-history timeline
Automated tests and seed data	Deployment as a mandatory requirement

4. Ticket Data Model

Field	Type / Allowed Values	Rules
id	Generated identifier	Generated automatically
title	String	Required; maximum 120 characters
description	Text	Required
customerEmail	Email string	Required; must be valid email format
priority	LOW MEDIUM HIGH	Required; UI may preselect Medium as an implementation assumption
status	OPEN IN_PROGRESS RESOLVED	Required; defaults to OPEN
createdAt	Timestamp	Generated automatically
updatedAt	Timestamp	Updated automatically when the ticket changes

Assumption: the specification does not define a default priority. If Medium is preselected in the UI, that choice must be documented as an implementation assumption rather than presented as a requirement.

5. Functional Requirements

FR-01 - Create Ticket

- The user can create a support ticket with title, description, customer email, and priority.
- Status defaults to Open.
- Created and updated timestamps are generated automatically.
- A successful creation persists in the database and remains after refresh.

FR-02 - Validate Input

- Frontend validation gives immediate, useful feedback.
- Backend validation independently enforces the same business rules.
- Invalid requests return meaningful HTTP status codes and a consistent error shape.

FR-03 - List Tickets

- The dashboard displays tickets in pages of 10.
- Each row/card shows enough context to identify the ticket quickly: title, customer email, status, priority, and created/updated time.

FR-04 - Search Tickets

- Search matches ticket title OR customer email.
- Search is handled by the backend and can be combined with filters, sorting, and pagination.

FR-05 - Filter and Sort

- Filter by status: Open, In Progress, Resolved.
- Filter by priority: Low, Medium, High.
- Sort by creation date: newest first or oldest first.

- All query controls work together.

FR-06 - View Ticket Details

- The user can open a ticket and view complete ticket details.

FR-07 - Update Ticket

- The user can update ticket status and priority.
- Changes persist after refresh.
- updatedAt changes automatically when an update succeeds.

FR-08 - Summary Counts

- Show Total, Open, In Progress, and Resolved counts.
- Counts represent the entire dataset and do not change when list filters or search are active.

6. Core User Flows

Flow A - Create a ticket

Dashboard -> New Ticket -> Enter details -> Frontend validation -> Submit -> Backend validation -> Persist -> Success confirmation -> Ticket appears in list

Flow B - Find a ticket

Dashboard -> Search and/or filters -> Backend query -> Paginated result -> Open ticket detail

Flow C - Update a ticket

Ticket detail -> Change status/priority -> Save -> Backend validates -> Persist update -> Success feedback -> Refreshed detail remains updated

7. Dashboard Requirements

The main dashboard must include:

- Header with product identity and New Ticket action.
- Summary cards: Total, Open, In Progress, Resolved.
- Search input for title or customer email.
- Status filter, priority filter, and creation-date sort.
- Ticket list/table with clear status and priority labels.
- Pagination controls for 10 tickets per page.
- Responsive behavior: table-style layout may become card-style layout on small screens.

Optional enhancement - Needs Attention

A derived visual indicator may be shown when a ticket is both High priority and Open. This is not a new database field and is not part of the mandatory assignment requirements. It is a lightweight UX enhancement supporting the product goal of finding requests that need attention.

8. Ticket Detail Requirements

- Display ticket ID, title, description, customer email, priority, status, created timestamp, and updated timestamp.
- Allow editing of status and priority only, because those are the required update fields.

- Provide a clear Save action and success/error feedback.
- Provide navigation back to the ticket list.

9. UI States and Responsive Behavior

State	Expected behavior
Loading	Show skeletons or a clear loading indicator while data is requested.
Empty dataset	Explain that no tickets exist and provide a Create Ticket action.
No search/filter matches	Explain that no tickets match and provide Clear Filters.
API/network error	Show a useful message and a Retry action.
Success	Confirm ticket creation/update with concise feedback.

10. API Behavior and Query Contract

The PRD does not mandate endpoint names; the following is the chosen implementation contract.

Method	Endpoint	Purpose
POST	/api/tickets	Create a ticket
GET	/api/tickets	List/search/filter/sort/paginate tickets
GET	/api/tickets/:id	Get one ticket
PATCH	/api/tickets/:id	Update status and/or priority
GET	/api/tickets/summary	Get unfiltered summary counts

Example backend-driven list query

```
GET /api/tickets?search=payment&status;=OPEN&priority;=HIGH&sort;=newest&page;=1&limit;=10
```

Expected list response shape

```
{ "success": true, "data": { "tickets": [...], "pagination": { "page": 1, "limit": 10, "total": 32, "totalPages": 4 } } }
```

Expected validation error shape

```
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "Invalid ticket data", "details": { "customerEmail": "Please provide a valid email address" } } }
```

11. Non-Functional Requirements

Area	Requirement
Responsiveness	Usable on desktop and mobile.
Persistence	Created and updated tickets remain after refresh.
Maintainability	Clear separation of routing, validation, service/business logic, and persistence.
Error handling	Meaningful HTTP status codes and consistent API errors.
Testability	At least three meaningful automated tests.
Seed data	At least 25 tickets with varied status and priority values.

Area	Requirement
Documentation	README includes setup, environment variables, database/seed steps, tests, choices, assumptions, limitations, and time spent.

12. Testing Requirements

Minimum automated test set

- Validation test: invalid customer email is rejected.
- Query test: combined status and priority filters return only matching records.
- Update test: status/priority update persists successfully.

Useful additional tests if time permits

- Title longer than 120 characters returns a validation error.
- Search matches both title and customer email.
- Pagination returns a maximum of 10 tickets per page.
- Summary counts remain independent of list filters.

13. Technical Implementation Summary

Chosen stack

Layer	Technology	Reason
Frontend	React + Vite	Fast setup; suitable for interactive dashboard UI.
Routing	React Router	Simple dashboard/detail navigation.
Styling	Tailwind CSS	Fast responsive styling and consistent UI states.
HTTP client	Axios or fetch	Simple REST integration.
Backend	Node.js + Express	Lightweight, familiar REST API framework.
Validation	Zod	Readable centralized request validation.
Database	PostgreSQL	Reliable persistent relational storage.
ORM	Prisma	Fast migrations, seed data, and type-safe querying.
Tests	Vitest/Jest + Supertest	Meaningful API and validation coverage.
Version control	Git + GitHub	Repository submission and change history.

Architecture style

Use a modular monolith. For a six-hour assignment with one main domain entity, microservices would add complexity without improving the required workflow.

Logical request flow

Browser / React -> REST API -> Express route/controller -> validation -> service layer -> Prisma -> PostgreSQL

14. Deployment and Environment

Required delivery mode

The application must run locally using documented setup instructions. Local execution is the mandatory acceptance path.

Optional deployment bonus

If the required application is complete and time remains, a live demo may use: frontend on Vercel, backend on Render, and PostgreSQL on Neon. Deployment is optional and must not compromise the local setup.

15. Assumptions and Tradeoffs

- There is one support-team user type; authentication is not implemented.
- Priority has no specified default in the assignment; if Medium is preselected, it is an implementation assumption.
- Only status and priority are editable after ticket creation because those are the update fields explicitly required.
- A ticket history/audit log is not implemented because it is outside the required scope.
- Summary metrics use the full dataset, while the list endpoint may be filtered and paginated.
- Optional visual enhancements are added only after all mandatory functionality is complete.

16. Acceptance Criteria

ID	Acceptance criterion
AC-01	A valid ticket can be created and persists after page refresh.
AC-02	Invalid title, description, or email input is rejected with useful feedback on both frontend and backend.
AC-03	Search matches title or customer email.
AC-04	Status and priority filters work individually and together.
AC-05	Newest/oldest sorting is handled by the backend.
AC-06	Ticket listing is paginated at 10 tickets per page by the backend.
AC-07	A ticket detail page shows complete ticket information.
AC-08	Status and priority can be updated and remain updated after refresh.
AC-09	Total/Open/In Progress/Resolved summary counts reflect the full dataset, regardless of active filters.
AC-10	Loading, empty, and error states are implemented.
AC-11	The interface is usable on desktop and mobile.
AC-12	Seed data contains at least 25 varied tickets.
AC-13	At least three meaningful automated tests pass.
AC-14	README documents setup, environment variables, seed/migration steps, tests, choices, assumptions, limitations, and time spent.

17. Requirement Traceability

Assignment requirement	PRD section
Create ticket + field rules	Sections 4-5 (FR-01, FR-02)

Assignment requirement	PRD section
Search/filter/sort/pagination	Sections 5, 7, 10
View/update ticket	Sections 5, 8
Summary counts	FR-08, Section 7
Responsive UI + UI states	Section 9
HTTP/error consistency	Sections 10-11
Automated tests	Section 12
25+ seed tickets	Section 11
README/setup documentation	Section 11 and AC-14
Local execution	Section 14
6-hour scope discipline	Sections 3 and 15

Final implementation rule

This PRD is frozen for the assignment. Any feature not required here should be treated as optional and should only be added after all acceptance criteria are working, tested, and documented.